

GUÍA COMPLETA

Generador de Paletas de Colores

Explicación línea por línea para principiantes

HTML + CSS + JavaScript

CONTENIDO

Sección	Página
1. HTML (Estructura)	3
2. CSS (Estilos visuales)	8
3. JavaScript (Lógica)	15
4. Conceptos clave	25

HTML - index.html

Este archivo define la **estructura** de la página. Es el esqueleto donde todo se coloca.

Código:

```
<!DOCTYPE html>
```

Explicación: Declara que este documento usa HTML5 (la versión moderna).

Código:

```
<html lang="es">
```

Explicación: Inicia la página. **lang="es"** indica que el contenido está en español (ayuda a lectores de pantalla y motores de búsqueda).

Código:

```
<head>
```

Explicación: Contiene metadatos: información sobre la página que NO es visible al usuario.

Código:

```
<meta charset="UTF-8" />
```

Explicación: Permite usar caracteres especiales: tildes, eñes, emojis, símbolos, etc.

Código:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

Explicación: Hace que la página se vea bien en celulares. **width=device-width** = el ancho de la página es el ancho de la pantalla del dispositivo.

Código:

```
<title>Generador de Paletas</title>
```

Explicación: Texto que aparece en la pestaña del navegador y en los marcadores.

Código:

```
<link rel="stylesheet" href="./style.css" />
```

Explicación: Conecta el archivo CSS (los estilos visuales). **rel="stylesheet"** = "esto es una hoja de estilos".

Código:

```
<script src="./app.js" defer></script>
```

Explicación: Conecta el archivo JavaScript (la lógica). **defer** = "ejecuta esto DESPUÉS de que todo el HTML cargue completamente".

Código:

```
</head>
```

Explicación: Cierra la sección de metadatos.

Código:

```
<body>
```

Explicación: Abre el cuerpo de la página: aquí va TODO lo visible.

Código:

```
<main class="app">
```

Explicación: <main> = contenido principal de la página. **class="app"** = nombre para aplicar estilos CSS.

Código:

```
<section class="hero" aria-labelledby="titulo">
```

Explicación: <section> = agrupa contenido relacionado. **class="hero"** = sección destacada.
aria-labelledby="titulo" = accesibilidad: le dice a lectores de pantalla que el título es el elemento con id="titulo".

Código:

```
<h1 id="titulo">Generador de paletas</h1>
```

Explicación: <h1> = título principal (el más importante). **id="titulo"** = identificador único que usa aria-labelledby arriba.

Código:

```
<p>Creá paletas aleatorias mostrando códigos HEX y HSL.</p>
```

Explicación: <p> = párrafo de texto descriptivo.

Código:

```
<form class="controls">
```

Explicación: <form> = agrupa controles interactivos. Aquí no envía datos a servidor, solo organiza visualmente.

Código:

```
<label for="paletteSize">Tamaño de la paleta</label>
```

Explicación: <label> = etiqueta descriptiva. **for="paletteSize"** = asociada al elemento con id="paletteSize". Hacer clic en el texto activa el menú desplegable.

Código:

```
<select id="paletteSize" name="paletteSize">
```

Explicación: <select> = menú desplegable. **id="paletteSize"** = identificador que usa el label y JavaScript.

Código:

```
<option value="6">6 colores</option>
```

Explicación: <option> = opción del menú. **value="6"** = el valor que JavaScript leerá. "6 colores" = texto visible al usuario.

Código:

```
<option value="8">8 colores</option>
```

Explicación: Otra opción: paleta de 8 colores.

Código:

```
<option value="9">9 colores</option>
```

Explicación: Otra opción: paleta de 9 colores.

Código:

```
</select>
```

Explicación: Cierra el menú desplegable.

Código:

```
<button type="button" id="generateBtn">
```

Explicación: <button> = botón clickeable. **type="button"** = no envía formulario, solo ejecuta JavaScript. **id="generateBtn"** = identificador para JavaScript.

Código:

```
Generar paleta
```

Explicación: Texto visible en el botón.

Código:

```
</button>
```

Explicación: Cierra el botón.

Código:

```
</form>
```

Explicación: Cierra el formulario.

Código:

```
</section>
```

Explicación: Cierra la sección hero.

Código:

```
<section aria-labelledby="paletaTitulo">
```

Explicación: Nueva sección para mostrar la paleta actual.

Código:

```
<h2 id="paletaTitulo">Paleta actual</h2>
```

Explicación: <h2> = subtítulo. **id="paletaTitulo"** = para accesibilidad (aria-labelledby).

Código:

```
<div id="palette" class="palette"></div>
```

Explicación: <div> = contenedor vacío. JavaScript llenará esto dinámicamente con los colores.
id="palette" = lo usa JavaScript para insertar colores.

Código:

```
</section>
```

Explicación: Cierra sección de paleta actual.

Código:

```
<section aria-labelledby="guardadasTitulo">
```

Explicación: Sección para las paletas guardadas.

Código:

```
<h2 id="guardadasTitulo">Paletas guardadas</h2>
```

Explicación: Título de la sección de paletas guardadas.

Código:

```
<button type="button" id="saveBtn">Guardar paleta</button>
```

Explicación: Botón para guardar la paleta actual en el almacenamiento local.

Código:

```
<div id="savedPalettes" class="saved-palettes"></div>
```

Explicación: Contenedor vacío donde JavaScript mostrará las paletas guardadas.

Código:

```
</section>
```

Explicación: Cierra sección de paletas guardadas.

Código:

```
</main>
```

Explicación: Cierra el contenido principal.

Código:

```
<div id="toast" class="toast" role="status" aria-live="polite"></div>
```

Explicación: <div> = notificación flotante (invisible al inicio). **role="status"** = accesibilidad: "esto es un mensaje de estado". **aria-live="polite"** = anuncia cambios sin interrumpir al usuario.

Código:

```
</body>
```

Explicación: Cierra el cuerpo visible de la página.

Código:

```
</html>
```

Explicación: Cierra el documento HTML.

CSS - style.css

Este archivo define los **estilos visuales**: colores, tamaños, posiciones, animaciones. Es la piel de la página.

Código:

```
* { box-sizing: border-box; }
```

Explicación: Selecciona TODOS los elementos de la página. **box-sizing: border-box** = el padding y el borde NO aumentan el tamaño total del elemento. Esto evita cálculos complicados de anchos.

Código:

```
body { margin: 0; font-family: Arial, Helvetica, sans-serif; background: #111827; color: #f9fafb; }
```

Explicación: Aplica a toda la página. **margin: 0** = quita el margen por defecto del navegador. **font-family** = tipografías (usa la primera disponible). **background** = color de fondo oscuro. **color** = color de texto blanco.

Código:

```
.app { width: min(1100px, 92%); margin: 0 auto; padding: 48px 0; }
```

Explicación: Selecciona elementos con class="app". **width: min(1100px, 92%)** = ancho: el MENOR entre 1100px o el 92% de la pantalla (responsive). **margin: 0 auto** = centrado horizontal. **padding: 48px 0** = espacio arriba y abajo, nada a los lados.

Código:

```
.hero { background: #1f2937; padding: 32px; border-radius: 24px; margin-bottom: 32px; }
```

Explicación: Sección del título y controles. **background** = gris oscuro. **padding** = espacio interno. **border-radius** = esquinas redondeadas. **margin-bottom** = espacio debajo.

Código:

```
h1, h2, p { margin-top: 0; }
```

Explicación: Selecciona h1, h2 y párrafos a la vez. **margin-top: 0** = quita el margen superior por defecto (evita espacios raros entre elementos).

Código:

```
h1 { font-size: 40px; margin-bottom: 8px; }
```

Explicación: Tamaño grande (40px) para el título principal. **margin-bottom** = pequeño espacio debajo del título.

Código:

```
p { color: #d1d5db; }
```

Explicación: Los párrafos se muestran en gris claro, menos prominentes que el título.

Código:

```
.controls { display: flex; align-items: end; gap: 16px; flex-wrap: wrap; margin-top: 24px; }
```

Explicación: Contenedor del select y botón. **display: flex** = organiza elementos en fila horizontal.

align-items: end = alinea los elementos abajo. **gap** = espacio entre elementos. **flex-wrap: wrap** = si no caben, bajan a otra línea.

Código:

```
label { display: block; margin-bottom: 8px; font-weight: bold; }
```

Explicación: **display: block** = el label ocupa toda la línea (no comparte espacio). **font-weight: bold** = texto en negrita.

Código:

```
select, button { min-height: 44px; border-radius: 12px; border: 2px solid transparent; font-size: 16px; }
```

Explicación: Estilos compartidos para select y botón. **min-height: 44px** = altura mínima (fácil de tocar en celulares). **border: 2px solid transparent** = borde invisible que se verá al enfocar con teclado.

Código:

```
select { padding: 0 14px; background: #ffffff; color: #111827; }
```

Explicación: Estilos específicos del menú desplegable: fondo blanco y texto oscuro.

Código:

```
button { cursor: pointer; padding: 0 18px; background: #6366f1; color: white; font-weight: bold; transition: transform 0.2s, background 0.2s; }
```

Explicación: **cursor: pointer** = cambia el cursor a manito al pasar el mouse. **transition** = animación suave cuando cambian las propiedades (color, posición).

Código:

```
button:hover { background: #4f46e5; transform: translateY(-2px); }
```

Explicación: **:hover** = cuando el mouse está encima del botón. Cambia a un morado más oscuro y sube 2px (efecto de flotar).

Código:

```
button:focus-visible, select:focus-visible, .color-card:focus-visible { outline: 3px solid #facc15; outline-offset: 4px; }
```

Explicación: **:focus-visible** = cuando navegas con teclado (tecla Tab). **outline** = borde amarillo de enfoque. **outline-offset** = separación del borde respecto al elemento.

Código:

```
.palette { display: grid; grid-template-columns: repeat(3, 1fr); gap: 18px; margin: 20px 0 40px; }
```

Explicación: Contenedor de los colores. **display: grid** = sistema de cuadrícula. **repeat(3, 1fr)** = 3 columnas de igual ancho. **1fr** = una fracción del espacio disponible.

Código:

```
.color-card { min-height: 160px; border-radius: 20px; padding: 16px; display: flex; align-items: end; justify-content: space-between; border: 3px solid rgba(255, 255, 255, 0.25); cursor: pointer; transition: transform 0.25s, box-shadow 0.25s; animation: fadeIn 0.25s ease; }
```

Explicación: Cada color de la paleta. **align-items: end** = contenido abajo. **justify-content: space-between** = código a la izquierda, candado a la derecha. **rgba(255,255,255,0.25)** = blanco semi-transparente (25% opacidad). **animation: fadeIn** = usa la animación definida más abajo.

Código:

```
.color-card:hover { transform: scale(1.03); box-shadow: 0 12px 28px rgba(0, 0, 0, 0.35); }
```

Explicación: Efecto hover: agranda la tarjeta un 3% y agrega sombra para efecto de profundidad.

Código:

```
.color-code { background: rgba(0, 0, 0, 0.65); color: white; padding: 8px 10px; border-radius: 10px; display: flex; flex-direction: column; gap: 4px; }
```

Explicación: Caja negra semi-transparente con los códigos HEX y HSL. **flex-direction: column** = uno debajo del otro.

Código:

```
.color-code strong { font-size: 15px; }
```

Explicación: El código HEX (en negrita) con tamaño 15px.

Código:

```
.color-code small { font-size: 11px; opacity: 0.9; }
```

Explicación: El código HSL (en texto pequeño) ligeramente transparente.

Código:

```
.lock-btn { background: rgba(0, 0, 0, 0.65); padding: 8px 10px; min-height: auto; }
```

Explicación: Botón del candado. **min-height: auto** = anula el 44px general para que sea más compacto.

Código:

```
.lock-btn:hover { background: rgba(0, 0, 0, 0.85); }
```

Explicación: Más oscuro al pasar el mouse sobre el candado.

Código:

```
.saved-palettes { display: flex; flex-direction: column; gap: 14px; margin-top: 18px; }
```

Explicación: Contenedor de paletas guardadas: una debajo de otra (columna).

Código:

```
.saved-palette { display: flex; overflow: hidden; border-radius: 14px; border: 2px solid #374151; }
```

Explicación: Cada paleta guardada. **display: flex** = colores en fila horizontal. **overflow: hidden** = recorta lo que sobresalga de las esquinas redondeadas.

Código:

```
.saved-color { height: 48px; flex: 1; }
```

Explicación: Cada franja de color de una paleta guardada. **flex: 1** = ocupa partes iguales del ancho total.

Código:

```
.toast { position: fixed; right: 24px; bottom: 24px; background: #f9fafb; color: #111827; padding: 14px 18px; border-radius: 12px; font-weight: bold; opacity: 0; transform: translateY(12px); pointer-events: none; transition: opacity 0.25s, transform 0.25s; }
```

Explicación: Notificación flotante. **position: fixed** = siempre visible, no se mueve al hacer scroll. **opacity: 0** = invisible inicialmente. **transform: translateY(12px)** = desplazado 12px hacia abajo. **pointer-events: none** = no interfiere con los clics del usuario.

Código:

```
.toast.show { opacity: 1; transform: translateY(0); }
```

Explicación: Cuando tiene la clase "show" (agregada por JavaScript), aparece y sube a su posición final.

Código:

```
@keyframes fadeIn { from { opacity: 0; transform: translateY(8px); } to { opacity: 1; transform: translateY(0); } }
```

Explicación: Define la animación "fadeIn": empieza invisible y bajito, termina visible y en posición normal.

Código:

```
@media (max-width: 768px) { ... }
```

Explicación: Media query: cambia estilos cuando la pantalla mide 768px o menos. Reduce tamaños y pasa a 2 columnas.

Código:

```
@media (max-width: 480px) { ... }
```

Explicación: Para pantallas muy pequeñas (celulares): 1 columna, botones al 100% de ancho, toast centrado.

JavaScript - app.js

Este archivo contiene la **lógica**: genera colores aleatorios, convierte formatos, guarda en memoria, responde a clics. Es el cerebro de la aplicación.

Código:

```
const paletteContainer = document.getElementById("palette");
```

Explicación: Obtiene el div id="palette" del HTML y lo guarda en una variable constante. **const** = no se puede reasignar esta variable.

Código:

```
const paletteSizeSelect = document.getElementById("paletteSize"); const generateBtn = document.getElementById("generateBtn"); const saveBtn = document.getElementById("saveBtn"); const savedPalettesContainer = document.getElementById("savedPalettes"); const toast = document.getElementById("toast");
```

Explicación: Obtiene más elementos del HTML para usarlos después en el código. Cada uno se guarda en una variable.

Código:

```
let currentPalette = [];
```

Explicación: **let** = variable que puede cambiar su valor. **currentPalette** = array vacío que guardará los colores actuales de la paleta.

Código:

```
function generateRandomHSL() {
```

Explicación: Declara una función (bloque de código reutilizable) que genera un color HSL aleatorio.

Código:

```
const hue = Math.floor(Math.random() * 360);
```

Explicación: **Math.random()** = número decimal entre 0 y 0.999... * **360** = entre 0 y 359.999...

Math.floor() = redondea hacia abajo (0 a 359). **hue** = matiz (el color en el círculo cromático).

Código:

```
const saturation = Math.floor(Math.random() * 31) + 60;
```

Explicación: * **31** = número entre 0 y 30. + **60** = entre 60 y 90. Saturación entre 60% y 90% (colores vivos, no grises).

Código:

```
const lightness = Math.floor(Math.random() * 31) + 40;
```

Explicación: Luminosidad entre 40% y 70% (ni muy oscuro ni muy claro, legible).

Código:

```
return `hsl(${hue}, ${saturation}%, ${lightness}%)`;
```

Explicación: **return** = devuelve el resultado al lugar que llamó la función. Template literal (backticks) = permite insertar variables dentro del texto con **\${}**.

Código:

```
function hslToHex(hsl) {
```

Explicación: Convierte un color de formato HSL a formato HEX (ej: #FF5733).

Código:

```
const values = hsl.match(/\d+/g).map(Number);
```

Explicación: **hsl.match(/\d+/g)** = encuentra todos los números en el string HSL. **.map(Number)** = convierte cada texto numérico a número real.

Código:

```
const h = values[0] / 360; const s = values[1] / 100; const l = values[2] / 100;
```

Explicación: Normaliza los valores: matiz a rango 0-1, saturación y luminosidad a rango 0-1.

Código:

```
let r, g, b;
```

Explicación: Declara variables para los componentes Rojo, Verde y Azul (RGB).

Código:

```
if (s === 0) { r = g = b = l; } else {
```

Explicación: Si la saturación es 0, es un tono de gris (todos los canales iguales a la luminosidad).

Código:

```
const hueToRgb = (p, q, t) => {
```

Explicación: Función interna que calcula componentes RGB (algoritmo estándar de conversión HSL a RGB).

Código:

```
if (t < 0) t += 1; if (t > 1) t -= 1;
```

Explicación: Normaliza el valor t al rango 0-1 (circular).

Código:

```
if (t < 1 / 6) return p + (q - p) * 6 * t; if (t < 1 / 2) return q; if (t < 2 / 3) return p + (q - p) * (2 / 3 - t) * 6; return p;
```

Explicación: Fórmulas matemáticas del estándar de conversión HSL a RGB.

Código:

```
const q = l < 0.5 ? l * (1 + s) : l + s - l * s; const p = 2 * l - q;
```

Explicación: `?` : = operador ternario (if corto). Calcula valores intermedios para la fórmula RGB.

Código:

```
r = hueToRgb(p, q, h + 1 / 3); g = hueToRgb(p, q, h); b = hueToRgb(p, q, h - 1 / 3);
```

Explicación: Aplica la fórmula a cada canal de color, desplazando el matiz para rojo, verde y azul.

Código:

```
const toHex = (color) => { const hex = Math.round(color * 255).toString(16); return hex.length === 1 ? "0" + hex : hex; };
```

Explicación: Convierte un valor 0-1 a 0-255, luego a hexadecimal. Si es un solo dígito (ej: "f"), le agrega un cero al inicio ("0f").

Código:

```
return `#${toHex(r)}${toHex(g)}${toHex(b)}`.toUpperCase();
```

Explicación: Une todo con # al inicio y convierte a mayúsculas (formato estándar HEX como #FF5733).

Código:

```
function showToast(message) {
```

Explicación: Muestra una notificación flotante en la esquina inferior derecha.

Código:

```
toast.textContent = message;
```

Explicación: Pone el texto recibido dentro del elemento toast.

Código:

```
toast.classList.add("show");
```

Explicación: Agrega la clase CSS "show" que hace visible la notificación.

Código:

```
setTimeout(() => { toast.classList.remove("show"); }, 1800);
```

Explicación: **setTimeout** = ejecuta el código después de 1800 milisegundos (1.8 segundos). Quita la clase y la notificación se oculta.

Código:

```
function generatePalette() {
```

Explicación: Genera una nueva paleta de colores aleatorios.

Código:

```
const size = Number(paletteSizeSelect.value);
```

Explicación: Obtiene el valor seleccionado del menú (6, 8 o 9) y lo convierte a número.

Código:

```
const newPalette = [];
```

Explicación: Array temporal para construir la nueva paleta.

Código:

```
for (let i = 0; i < size; i++) {
```

Explicación: Bucle que se repite "size" veces (ej: 6 veces para 6 colores).

Código:

```
if (currentPalette[i]?.locked) {
```

Explicación: ?. = optional chaining. Si existe el color en esa posición y está bloqueado...

Código:

```
newPalette.push(currentPalette[i]);
```

Explicación: ...copia el color existente (no lo cambia, respeta el bloqueo).

Código:

```
} else { const hsl = generateRandomHSL(); const hex = hslToHex(hsl);
```

Explicación: Si no está bloqueado, genera un nuevo color aleatorio y lo convierte a HEX.

Código:

```
newPalette.push({ hsl, hex, locked: false });
```

Explicación: Agrega un objeto con tres propiedades al array: hsl, hex, y estado de bloqueo.

Código:

```
currentPalette = newPalette; renderPalette(); showToast("Paleta generada");
```

Explicación: Guarda la nueva paleta, la dibuja en pantalla, y muestra mensaje de confirmación.

Código:

```
function renderPalette() {
```

Explicación: Dibuja/visualiza la paleta actual en el HTML.

Código:

```
paletteContainer.innerHTML = "";
```

Explicación: Limpia el contenedor (borra los colores anteriores para redibujar).

Código:

```
currentPalette.forEach((color, index) => {
```

Explicación: Por cada color en la paleta (recibe el color y su posición numérica).

Código:

```
const colorCard = document.createElement("article");
```

Explicación: Crea un elemento en memoria (aún no visible).

Código:

```
colorCard.className = "color-card"; colorCard.style.backgroundColor = color.hsl;  
colorCard.tabIndex = 0;
```

Explicación: Le asigna clase CSS, color de fondo, y lo hace enfocable con la tecla Tab.

Código:

```
colorCard.setAttribute("aria-label", `Color ${color.hex}. Click para copiar`);
```

Explicación: Accesibilidad: descripción para lectores de pantalla de personas con discapacidad visual.

Código:

```
const colorCode = document.createElement("span"); colorCode.className =  
"color-code"; colorCode.innerHTML = ` ${color.hex} ${color.hsl} `;
```

Explicación: Crea la caja con el código HEX (negrita) y HSL (pequeño) usando template literals.

Código:

```
const lockButton = document.createElement("button"); lockButton.className =  
"lock-btn"; lockButton.type = "button"; lockButton.textContent = color.locked ? "■"  
: "□";
```

Explicación: Crea el botón de candado. ? := si está bloqueado muestra candado cerrado, si no, abierto.

Código:

```
lockButton.setAttribute( "aria-label", color.locked ? "Desbloquear color" :  
"Bloquear color" );
```

Explicación: Texto descriptivo accesible para el botón de candado.

Código:

```
colorCard.addEventListener("click", () => { copyHex(color.hex); });
```

Explicación: Cuando el usuario hace clic en la tarjeta de color, copia el código HEX al portapapeles.

Código:

```
colorCard.addEventListener("keydown", (event) => { if (event.key === "Enter") {  
copyHex(color.hex); } });
```

Explicación: Cuando el usuario presiona Enter estando enfocado en la tarjeta, también copia el HEX.

Código:

```
lockButton.addEventListener("click", (event) => { event.stopPropagation();
```

Explicación: **stopPropagation()** = evita que el clic se propague a la tarjeta (no copiaría al hacer clic en el candado).

Código:

```
currentPalette[index].locked = !currentPalette[index].locked;
```

Explicación: **!** = invierte el valor booleano (true se convierte en false, y viceversa).

Código:

```
renderPalette(); showToast(currentPalette[index].locked ? "Color bloqueado" :  
"Color desbloqueado"); });
```

Explicación: Redibuja toda la paleta y muestra mensaje según el nuevo estado.

Código:

```
colorCard.appendChild(colorCode); colorCard.appendChild(lockButton);  
paletteContainer.appendChild(colorCard);
```

Explicación: Adopta los elementos: mete el código y el candado dentro de la tarjeta, y la tarjeta dentro del contenedor principal.

Código:

```
function copyHex(hex) { navigator.clipboard.writeText(hex); showToast(`${hex}  
copiado`); }
```

Explicación: **navigator.clipboard** = API del navegador para copiar texto al portapapeles del sistema.

Código:

```
function savePalette() {
```

Explicación: Guarda la paleta actual en el almacenamiento local del navegador.

Código:

```
if (currentPalette.length === 0) { showToast("Primero generá una paleta"); return;  
}
```

Explicación: Si no hay paleta generada, muestra error y sale de la función inmediatamente (**return**).

Código:

```
const savedPalettes = getSavedPalettes(); savedPalettes.push(currentPalette);
```

Explicación: Obtiene las paletas previamente guardadas y agrega la paleta actual al final.

Código:

```
localStorage.setItem("savedPalettes", JSON.stringify(savedPalettes));
```

Explicación: **localStorage** = almacenamiento persistente en el navegador (persiste al cerrar la pestaña).
JSON.stringify = convierte el array de objetos a texto plano.

Código:

```
renderSavedPalettes(); showToast("Paleta guardada"); }
```

Explicación: Muestra todas las paletas guardadas y notifica al usuario.

Código:

```
function getSavedPalettes() { return  
JSON.parse(localStorage.getItem("savedPalettes")) || []; }
```

Explicación: **JSON.parse** = convierte el texto de vuelta a array de objetos. **|| []** = si no hay nada guardado, devuelve array vacío (evita errores).

Código:

```
function renderSavedPalettes() {
```

Explicación: Dibuja las paletas guardadas en la sección inferior.

Código:

```
const savedPalettes = getSavedPalettes(); savedPalettesContainer.innerHTML = "";
```

Explicación: Obtiene las paletas guardadas y limpia el contenedor visual.

Código:

```
savedPalettes.forEach((palette) => { const paletteElement =  
document.createElement("div"); paletteElement.className = "saved-palette";
```

Explicación: Por cada paleta guardada, crea un contenedor div con clase CSS.

Código:

```
palette.forEach((color) => { const colorElement = document.createElement("div");  
colorElement.className = "saved-color"; colorElement.style.backgroundColor =  
color.hex; colorElement.title = color.hex;  
paletteElement.appendChild(colorElement); });
```

Explicación: Por cada color de esa paleta, crea una franja con el color de fondo y un tooltip con el código.

Código:

```
savedPalettesContainer.appendChild(paletteElement); }); }
```

Explicación: Mete la paleta completa dentro del contenedor principal de paletas guardadas.

Código:

```
generateBtn.addEventListener("click", generatePalette);  
saveBtn.addEventListener("click", savePalette);  
paletteSizeSelect.addEventListener("change", generatePalette);
```

Explicación: Conecta los botones y el select con sus funciones correspondientes. **change** = evento cuando cambia la selección del menú.

Código:

```
renderSavedPalettes(); generatePalette();
```

Explicación: Al cargar la página: muestra paletas guardadas previamente y genera una nueva paleta automáticamente.

CONCEPTOS CLAVE PARA TU FUTURO PROYECTO

Estos son los conceptos fundamentales que necesitas dominar para crear proyectos similares:

const vs let

const = variable que NO puede cambiar de valor. **let** = variable que SÍ puede cambiar. Usa const por defecto, let solo cuando necesites reasignar.

document.getElementById()

Traeme del HTML el elemento que tenga este id. Es la forma más común de conectar JavaScript con HTML.

addEventListener

Cuando pase X evento (clic, tecla, cambio), ejecuta Y función. Es cómo respondes a las acciones del usuario.

createElement / appendChild

Crear elementos HTML desde JavaScript y adoptarlos dentro de otros elementos. Permite generar contenido dinámico.

localStorage

Base de datos simple del navegador. Guarda texto que persiste incluso si cierras la pestaña. Máximo ~5MB.

JSON.stringify / JSON.parse

Convertir objetos/array de JavaScript a texto (para guardar) y de texto a objetos/array (para leer).

Template Literals

Strings con backticks que permiten insertar variables con **\${variable}**. Mucho más cómodo que concatenar con +.

Array Methods

.forEach() = por cada elemento haz algo. **.push()** = agrega al final. **.map()** = transforma cada elemento. **.filter()** = filtra elementos.

Optional Chaining ?.

Si la propiedad existe, accede a ella; si no, devuelve undefined sin explotar el programa. Útil para evitar errores.

CSS Grid / Flexbox

Flexbox = organiza elementos en UNA dimensión (fila o columna). **Grid** = organiza en DOS dimensiones (filas Y columnas).

Media Queries

Cambia estilos según el ancho de la pantalla. Esencial para que tu sitio se vea bien en celulares, tablets y computadoras.

Event Bubbling / stopPropagation

Los eventos de clic suben por los elementos padres. **stopPropagation()** detiene esa subida cuando no la quieres.

HSL vs HEX

HSL = Hue (matiz 0-360), Saturation (0-100%), Lightness (0-100%). Más intuitivo para humanos. **HEX** = formato #RRGGBB, más compacto para código.

Consejo final: Empieza por proyectos pequeños. Modifica este código: cambia colores, agrega un botón de eliminar paletas, o haz que los colores guardados sean clickeables. La práctica es la mejor forma de aprender!